

CONTRACT NOTE REWORK

Remaining Scope

Estimates 2-5 - Summary

~39.1 developer days across four estimates

About this document

This document summarises the remaining scope of the contract note rework, the four estimates that follow the delivered PDF / template management work (Estimate 1). It is the executive-level companion to the detailed per-estimate walkthroughs: enough to understand what each piece does, how it works, and what it costs, without the full technical depth.

The four estimates are largely independent and can be prioritised or sequenced separately. Each builds on the foundation delivered in Estimate 1 and, where noted, on each other.

The remaining estimates at a glance

Four estimates, ~39.1 developer days in total (~31.1 required plus ~8 optional testing). Days are shown split into required build and optional testing.

#	ESTIMATE	WHAT IT DELIVERS	REQUIRED	OPTIONAL	TOTAL
2	DocuSign Integration	Automated e-signature: sends the rendered PDF for signing and files the signed copy in Salesforce	4.1	3	7.1
3	Training & Data Sources	Enablement materials (3a) plus a capability to enrich contract notes with new data sources, no code change (3b)	10.4	2.5	12.9
4	Bespoke Contracts	A manual authoring flow for one-off, non-standard contract notes, reusing the existing editor and pipeline	4.3	2.5	6.8
5	Comparison Audit	A developer-operated tool that checks whether sent PDFs were edited after rendering	12.4	0	12.4
Total			31.1	8	39.1

How to read the effort figures

Totals include optional testing. The required build across the four is ~31.1 days; the ~8 optional days are property-based and integration tests that can be deferred for a faster MVP but are recommended, as these features handle live, customer-facing contracts.

Two estimates carry open questions that need BRYT confirmation (Estimates 2 and 5). Estimate 5 in particular has a hard external dependency on Microsoft 365 mailbox access.

Estimate 2 - DocuSign Integration

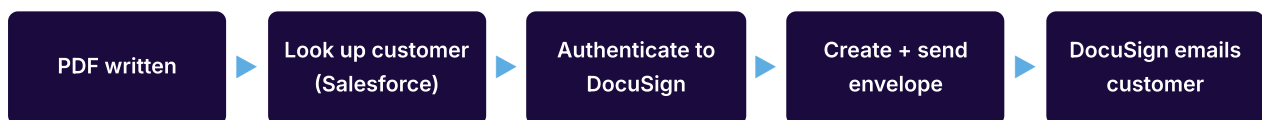
~7.1 developer days (4.1 required + testing).

Once a contract note PDF is produced, it still needs to be signed. Today that final step is manual: send the document, chase the signature, collect the signed copy, and file it.

Estimate 2 automates the whole loop through DocuSign.

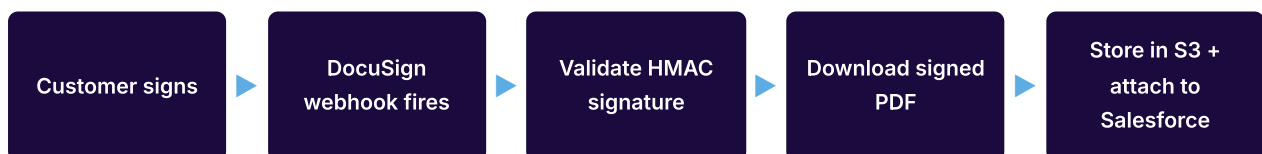
It is a headless, event-driven pipeline with no new Admin Portal screen. When Estimate 1's render pipeline writes a finished PDF, a new final step sends it to the customer for signature; when the customer signs, a webhook retrieves the signed copy and attaches it to their Salesforce record. The only visible outcome is a signed contract appearing in Salesforce.

Send phase (automatic, on PDF creation)



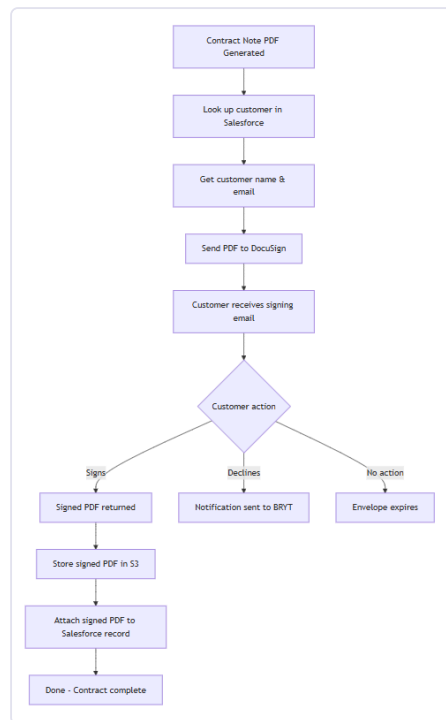
The customer receives the signing request as soon as the envelope is sent.

Completion phase (when the customer signs)



A declined or expired envelope instead writes a notification record for follow-up.

Simplified flow



Stakeholder-friendly view of the signing journey.

Key points

- › Two small Lambdas, with DocuSign and Salesforce either side; JWT auth means no human login.
- › Fails safe: a signing failure never fails the render or discards the finished PDF.
- › Needs a small Estimate 1 change: surface the customer's Salesforce reference through to the PDF output.
- › A new Salesforce client is built fresh (no existing one to reuse) for the customer lookup and file upload.
- › Open questions to confirm: single vs multi-party signing, whether BRYT has a DocuSign account, email branding, and the exact Salesforce object mapping.

Estimate 3 - Training & Data Sources

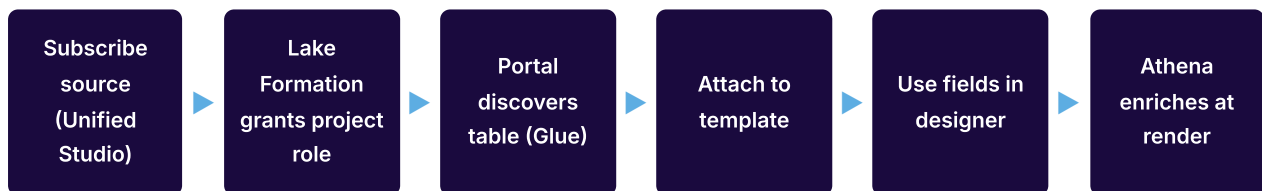
~12.9 developer days (3a: 4 + 3b: 8.9).

Estimate 3 has two independent parts that can be prioritised separately.

3a - Training & Enablement (~4 days) is a documentation effort, not software. The goal is to get a small team (2-5 people) productive on the template system without ongoing developer support: a quick-start guide, task-based how-tos, a data field reference, a rules cheat sheet, and a troubleshooting FAQ. Materials are drafted from the wireframes during the build and finalised with real screenshots afterwards.

3b - Data Source Extensibility (~8.9 days) is the build. Contract notes can today only show data that arrives in the contract payload; adding anything new has meant a developer change. 3b removes that dependency: business users subscribe a data source in SageMaker Unified Studio, attach it to a template, use its fields in the section designer, and at render time the pipeline fetches and merges the matching data, no code change or redeployment.

From subscription to rendered field (3b)



A newly subscribed data source is usable end-to-end with no code change.

Key points

- › Access is inherited by assuming the Unified Studio project role, so new subscriptions need no IAM changes.
- › Discovery uses the Glue Data Catalog; render-time enrichment uses Athena (serverless SQL).
- › The join key is the BrytNumber; only tables exposing a bryt_number column are offered.
- › Fields are namespaced as {source}.{column} to avoid collisions and make provenance clear.
- › No new table: attachment and dependency records are added to Estimate 1's existing DynamoDB table.

- Fails safe: a missing row renders empty with a warning; a failed Athena query halts (no partial PDF).

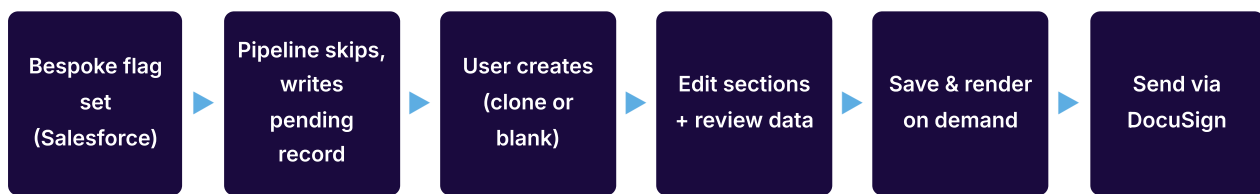
Estimate 4 - Bespoke Contracts

~6.8 developer days (4.3 required + testing).

Not every customer fits the standard mould, some need non-standard terms or VIP treatment, cases where automated template-matching is not the right tool. Today these are handled by manually editing a PDF: slow, error-prone, and with no audit trail.

Estimate 4 gives business users a proper way to produce these one-off documents. A customer is flagged as bespoke on their Salesforce record; the automated pipeline skips them and records a pending request. A user then composes, renders, and sends the document manually through a dedicated area of the Admin Portal, using the same section editor they already know.

End-to-end journey



The automated pipeline steps aside; a person drives the rest through the Admin Portal.

Key points

- › Built almost entirely from reuse: Estimate 1's section editor, shared sections, and render pipeline, plus Estimate 2's DocuSign flow triggered manually.
- › The recent template-preview feature already provides on-demand render-and-poll, so little new rendering machinery is needed.
- › Cloned sections are independent copies, bespoke edits never touch standard templates.
- › The pipeline skip is fail-safe: if Salesforce is unreachable, it renders as standard rather than blocking.
- › Every render is preserved in an append-only history; the current version is the one sent for signature.
- › One open question: confirm the Salesforce field that flags a customer as requiring a bespoke contract.

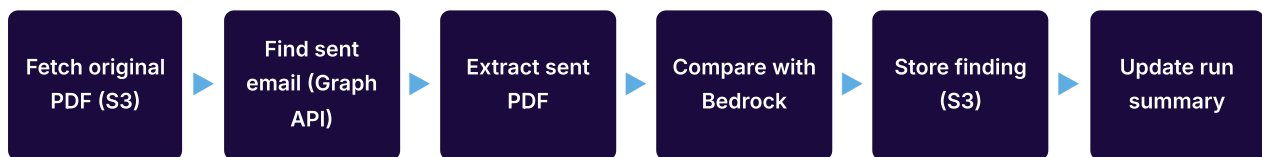
Estimate 5 - Comparison Audit

~12.4 developer days (incl. prompt iteration).

BRYT suspects some contract note PDFs may be edited by hand after rendering but before they reach the customer, sidestepping the controlled template system. Because the edit happens outside any system, there is currently no way to know whether or how often it occurs.

Estimate 5 is a developer-operated audit tool, not a customer- or business-facing product. For a batch of contract notes it retrieves the original rendered PDF (from S3) and the version actually emailed to the customer (via the Microsoft Graph API), compares the two with AWS Bedrock, and reports any differences. It runs ad-hoc (for example monthly), and findings are queried with Athena and delivered to BRYT as a spreadsheet.

Per-record comparison flow



If either PDF is missing, an error finding is recorded and the run continues.

Key points

- › A Step Function iterates the batch (up to 500 references, 5 at a time to respect Graph API limits).
- › The sent PDF comes from the actual email, the source of truth for what the customer received.
- › Bedrock gives a match/mismatch verdict plus a description of each difference; results land in S3 for Athena.
- › The comparison prompt lives in Parameter Store (versioned) so it can be tuned without redeploying.
- › A large share of the effort is prompt iteration and analysis rather than pure build; the spec notes a wider 13-20 day range depending on how many refinement cycles BRYT wants.

Critical dependency (Estimate 5)

The whole tool depends on read access to the mailbox where contract notes are sent, which requires BRYT's Microsoft 365 admin to grant an Azure AD app registration with Mail.Read (or Mail.ReadBasic). Without it, the sent PDFs cannot be retrieved and the tool cannot function. It is worth securing this early, as everything else is downstream of it. Two related questions, which mailbox(es) to search and how to correlate an email to a contract note, also need confirmation.

Open questions

Assumptions needing BRYT confirmation before or during build; each has a working assumption so delivery can proceed. Estimate 3 has none.

EST	#	QUESTION	WORKING ASSUMPTION / WHY IT MATTERS
2	1	Multi-party signing, or just the customer?	Single signatory assumed; multi-party changes envelope routing and order.
2	2	Does BRYT have an existing DocuSign account?	Assumed from scratch: new account, credentials, and sandbox.
2	3	Is DocuSign's branded email acceptable?	Yes; BRYT branding configured in DocuSign settings.
2	4	Is an Admin Portal UI needed for envelope status?	No UI this phase; status kept in metadata only.
2	5	Which Salesforce object, and is it REST-queryable?	Confirm the object and a REST/OAuth path before build.
2	6	Are voided envelopes, resend, and reminders in scope?	Out of scope; completed, declined, and expired only.
4	7	Should bespoke notes also go through DocuSign?	Yes; reuse Estimate 2, triggered manually rather than automatically.
4	8	How is a bespoke customer flagged?	A Salesforce field; the automated pipeline skips them.
5	9	Azure AD app with Mail.Read for the Graph API?	Critical dependency; without it the tool cannot function.
5	10	Which mailbox(es) are contract notes sent from?	Sets the search target; one shared mailbox or several users.
5	11	How do we correlate an email to a contract note?	Assumed the subject or filename carries the offer reference (e.g. OP-56412).